CALIFORNIA STATE UNIVERSITY, NORTHRIDGE

Edge Detection in FPGA: Sobel Filter Design

A graduate project submitted in partial fulfillment of the requirements

For the degree of Master of Science in Computer Engineering

By

Aaron Guadalupe Funes

December 2026

(Official end of the semester)

The graduate project of Aaron Funes is approved:

_____	_____
Professor Aaron Nanas	Date
_____	_____
Dr. Myung Cho	Date
_____	_____
Dr. Shahnam Mirzaei, Chair	Date

Table of Contents

Signature Page ii
List of Tables iv
List of Figures v
Acronym List vi
Abstract vii
Introduction 1
Design Implementation 4
Results 11
References 14

List of Tables

Table 1 : Bit Range Definition.....5

List of Figures

Figure 1 : Original Image 1
Figure 2 : Post-processed image with edge detection applied 1
Figure 3: X and Y Filter 2
Figure 4 : Filter Operation 2
Figure 5: Zybo Block Diagram[1] 3
Figure 6 : Grey Scale Range 3
Figure 7: Sobel Filter Block Design 4
Figure 8: Sobel Filter State Diagram 4
Figure 9: Sobel X & Y State Machine Diagram 5
Figure 10 : Overflow Logic 6
Figure 11: Convoluted Saturation and Overflow 6
Figure 12: BMP Data Type Structure 7
Figure 13 : Sobel Block Diagram 8
Figure 14 : SD Card Mount 9
Figure 15 : IP Address Writing 9
Figure 16: Done Polling 9
Figure 17 : Register Calls 10
Figure 18 : Destination Image Save 10
Figure 19 : Golden Model 11
Figure 20 : Max Pixel Value check 11
Figure 21 : Simulation Results 11
Figure 22 : Test Log Output 12
Figure 23 : UART Response 12
Figure 24 : Source Image 13
Figure 25 : Sobel Filter Applied Image 13

Acronym List

FPGA	Field-Programmable Gate Array
PL	Programmable Logic
AP SOC	All-Programmable System-On-Chip
PS	Processing System

Abstract

Edge Detection in FPGA: Sobel Filter Design

By

Aaron Guadalupe Funes

Master of Science in Computer Engineering

Edge detection algorithms are an important image processing technique used in our current world. They are used across many applications, medical devices, surveillance systems, and autonomous systems for example. The Sobel Filter is an edge detection algorithm that applies a 3x3 filter upon a subsection of an image to find any edges that may exist. When performed in software the filter is applied consecutively through the data of the image. With the use of a Field-Programmable Gate Array (FPGA), the filter could be applied in parallel allowing the edge detection process to be sped up. In this project, a greyscale Bitmap (BMP) image will be loaded to the Zybo Z7-20 development board through a micro SD card, where the filter design will exist in the programmable logic (PL), accessible through an IP block, have the filter applied to the image, and output the updated image with the edge detection applied. The Verilog design code for the sobel filter is verified through a simulation testbench that performs randomized self-checking. The initial C code written to read and write a BMP file set an expected baseline of how the code should work on the processor within the Zybo Z7 board. The Vitis IDE is used to load the final design to the Zybo Z7 board that saves the image to the SD card inserted. The results are then reviewed to determine the success of the filter application.

Introduction

Edge detection algorithms are an important image processing technique used in our current world. They are used across many applications, medical devices, surveillance systems, and autonomous systems for example. An example of edge detection is shown in Figure 1, the original image, and Figure 2, the post image-processing image.

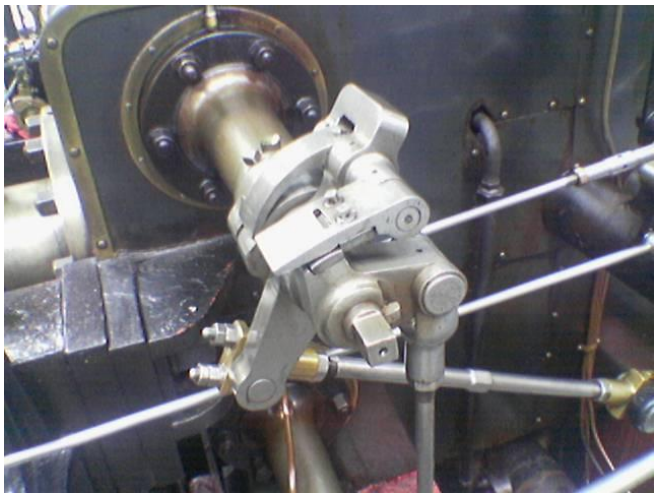


Figure 1 : Original Image

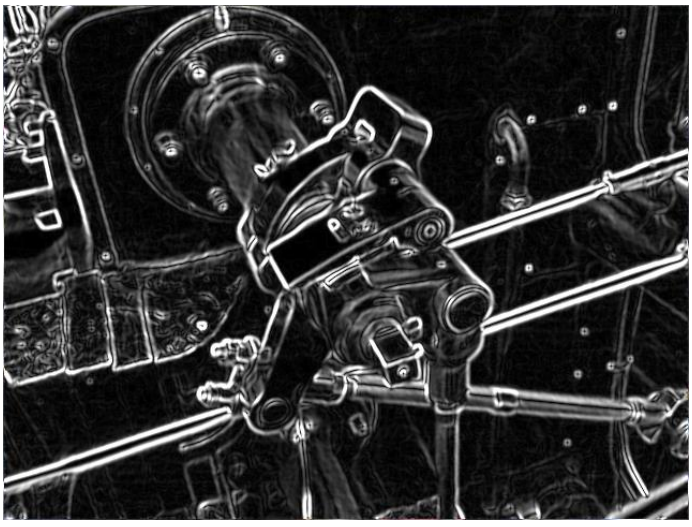


Figure 2 : Post-processed image with edge detection applied

The Sobel Filter is an edge detection algorithm that applies a 3x3 filter upon a subsection of an image to find any edges that may exist. The filter is performed by applying an X-direction derivative and a y-direction derivative, as shown in Figure *3*.

$$\mathbf{G}_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix} * \mathbf{A}$$

$$\mathbf{G}_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix} * \mathbf{A}$$

Figure 3: X and Y Filter

To apply the derivative, weights are multiplied to each pixel location with the upper row being pixels (0,0), (0,1), (0,2), where the designation is (row,column). Once all the weights are applied to each pixel location they are summed providing the associated output value for G_x or G_y.

Once the convoluted values are found for G_x and G_y, the following equation, shown in Figure 4, is performed to get the gradient magnitude at the current position of the filter in the image.

$$\mathbf{G} = \sqrt{\mathbf{G}_x{}^2 + \mathbf{G}_y{}^2}$$

Figure 4 : Filter Operation

The Zybo Z7-20 development board is used for the application of this project. The Zybo Z7 is a All Programmable System-on-Chip (AP SoC) device. It contains a dual-core ARM Cortex-A9 processor with Field Programmable Gate Array (FPGA) logic. The board can be broken down in to two sections, programmable logic and the processing system (PS).

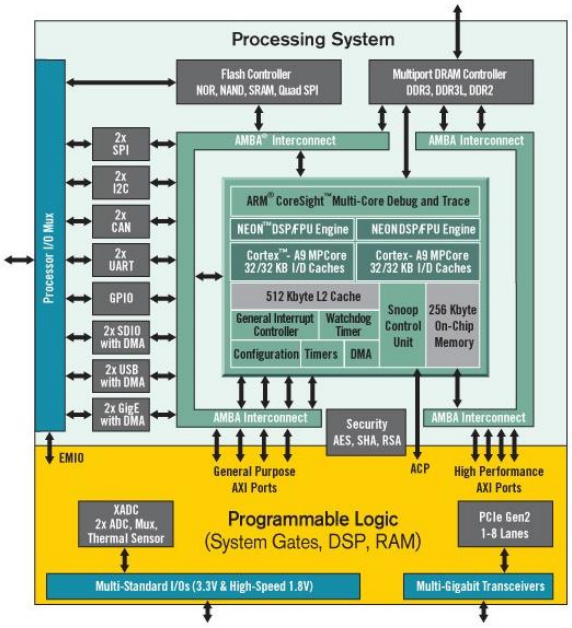


Figure 5: Zybo Block Diagram[1]

In order to access the PL design for the Sobel filter, Advanced eXtensible Interface (AXI) buses are used to establish communications with PS portion. The AXI buses will be used to read and write to addresses chosen as targets for the inputs and outputs of the PL.

To perform the arithmetic correctly in the firmware logic, signed fixed-point arithmetic is used. Using a grey-scale image each pixel is represented by a uint8, unsigned 8-bit, value. On the grey-scale range, a max value of 255 is white while a min value of zero is black.

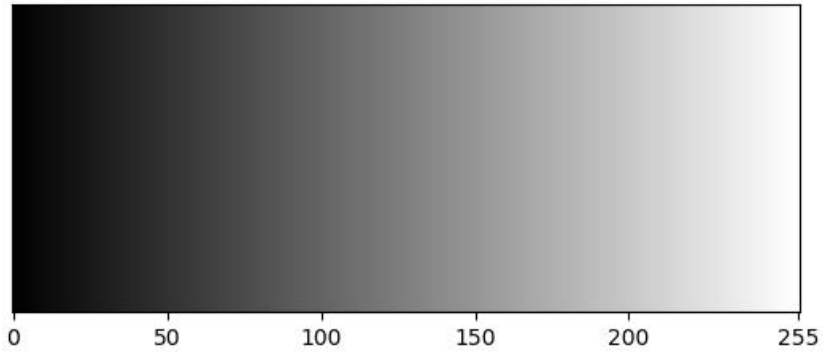


Figure 6 : Grey Scale Range

Design Implementation

The Sobel design was broken down into three submodules, as shown in Figure 7. The Sobel X module handles the x-derivative, the Y module handles the y-derivative, and the Square Root module to finalize the output.

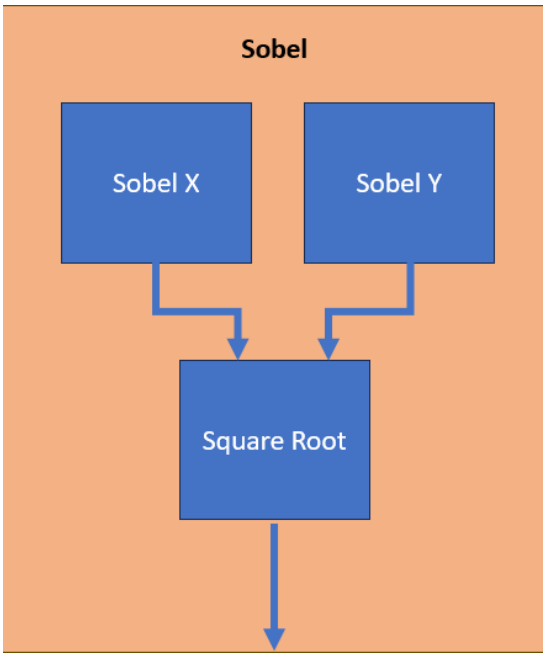


Figure 7: Sobel Filter Block Design

Finite-state machines (FSMs) control how the logic transitions through determined stages. The state machine below in Figure 9, is the state flow for the main Sobel filter module. A start signal is used to initiate the module state machine.

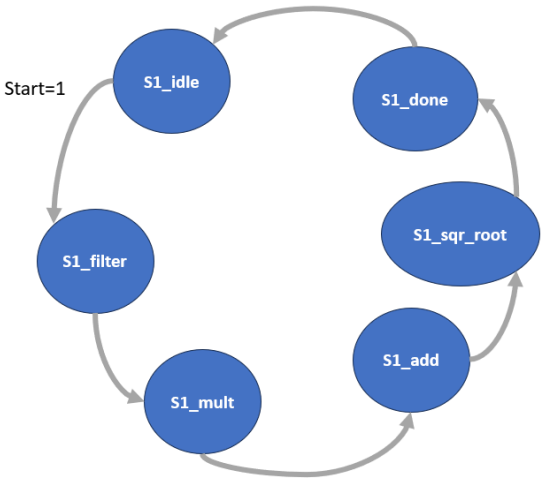


Figure 8: Sobel Filter State Diagram

The state machine below in Figure 9, is the state flow for the Sobel X and Y submodules. The arithmetic logic is broken down in to multiple stages to handle the saturation and overflow checks correctly.

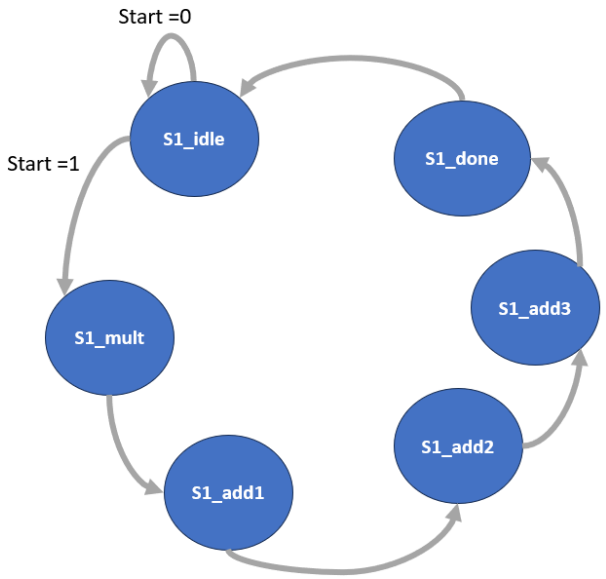


Figure 9: Sobel X & Y State Machine Diagram

The input array subsection for both the X and Y sub modules are nine uint8 values. To perform the convolution operation, the max and min ranges need to be determined for the correct bit range for G_x and G_y. The max range is a pixel value of 255 in all the positive weights and a pixel value of zero in every other weight, gaining a maximum value of 1020. This holds true for the minimum value with the negative weights, where the minimum value is -1020. For this to be stored properly in signed fixed point, a bit length of 11 is used. At the most significant bit (MSB) the weight is -2^{10}, the sign bit, giving a full range to -1024 to 1023.

-2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0
MSB										LSB

Table 1 : Bit Range Definition

For the multiplication stage a saturation check is added to keep the output value to minimum value of -510 and a positive value of 510. Once the weights are applied the summation stages are broken up into groups of two for the sign bits to be tracked properly.

```
s1_add_1: begin
    state <= s1_add_2 ;
    pixel_0_0 <= pixel_0_0;
    pixel_0_1 <= pixel_0_1;
    pixel_0_2 <= pixel_0_2;
    pixel_2_0 <= pixel_2_0;
    pixel_2_1 <= pixel_2_1;
    pixel_2_2 <= pixel_2_2;
    s_pixel_sum_1 <=  overflow_check_add(pixel_0_0 + pixel_0_1,11'h3FC,11'hC04); //check for overflow
    s_pixel_sum_2 <=  overflow_check_add(pixel_0_2 + pixel_2_0,11'h3FC,11'hC04);
    s_pixel_sum_3 <=  overflow_check_add(pixel_2_1 + pixel_2_2,11'h3FC,11'hC04);
    s_pixel_out <= s_pixel_out;
    pixel_out <= pixel_out;
end
```

Figure 10 : Overflow Logic

After the convoluted values are computed the max output from the squared values is 1,040,400. A saturation and overflow check are applied again in the Verilog code, shown in Figure 11, for square and sum calculations for the square root module.

```
s1_mult: begin
    g_sum <= g_sum;
    gx_pixel_squared <= sat_check_mult_sq(gx_pixel * gx_pixel,21'h0FE010,21'h101FF0);
    gy_pixel_squared <= sat_check_mult_sq(gy_pixel * gy_pixel,21'h0FE010,21'h101FF0);
    done <= done;
    start_filter <= start_filter;
    pixel_out <= pixel_out;
    start_sqrt <= start_sqrt;
    state <= s1_add;
end

s1_add: begin
    g_sum <= overflow_check_add_s22(gx_pixel_squared+gy_pixel_squared,22'h1FC020,22'h203FE0);
    gx_pixel_squared <= gx_pixel_squared;
    gy_pixel_squared <= gy_pixel_squared;
    done <= done;
    start_filter <= start_filter;
    pixel_out <= pixel_out;
    start_sqrt <= 1'b1;
    state <= s1_sqr_root;
end
```

Figure 11: Convoluted Saturation and Overflow

The square root module is based on the algorithm description described in the ICCD conference paper, A New Non-Restoring Square Root Algorithm and Its VLSI Implementations[2], by Yamin Li and Wanming Chu. The initial source code referenced is from verilogcodes.blogspot.com[3], posted by user vipin. In this square root design, the algorithm does iterations of add and subtracts and no use of multipliers.

The source code was updated with a state machine flow and start signal logic to initiate the computation. Once the square root value is calculated, a check determines if the value is greater than the max value of 255.

The file type chosen to hold the image data is BMP. BMP can store raw, uncompressed pixel data with no concern for losing quality in the image. To decode a BMP file, the header portion provides the relevant information about the image. The information available is the width, height, byte offset location for where the pixel data is located, and number of bytes per pixel. The baseline C code to read a BMP file is referenced from https://github.com/subrata05/image-processing-in-c/[4], written by user Subrata Sarkar. A structure holds the information provided by the header and the pixel values, shown in Figure 12.

```
22 typedef struct
23 {
24   //char is 1 byte
25   //array of char, size of 14
26     unsigned char fileHeader[BMP_FILE_HEADER_SIZE];
27     unsigned char *infoHeader;
28     uint32_t infoHeaderSize;
29
30     unsigned char *colorTab;
31     int colorTabSize;
32
33     int32_t width, height;
34     uint16_t bitDepth;
35     uint32_t offset;
36     unsigned char *pixel;
37 } BMPImage_t;
```

Figure 12: BMP Data Type Structure

The pixel value starting address location is assigned to the pointer variable, that is accessed to provide the pixel matrix needed for the Sobel operator.

The PL and PS communicate through an AXI IP block that wraps the Sobel Verilog code. The IP block is then instantiated in a block diagram with the other required block designs to communicate with the processor, as show in Figure 13.

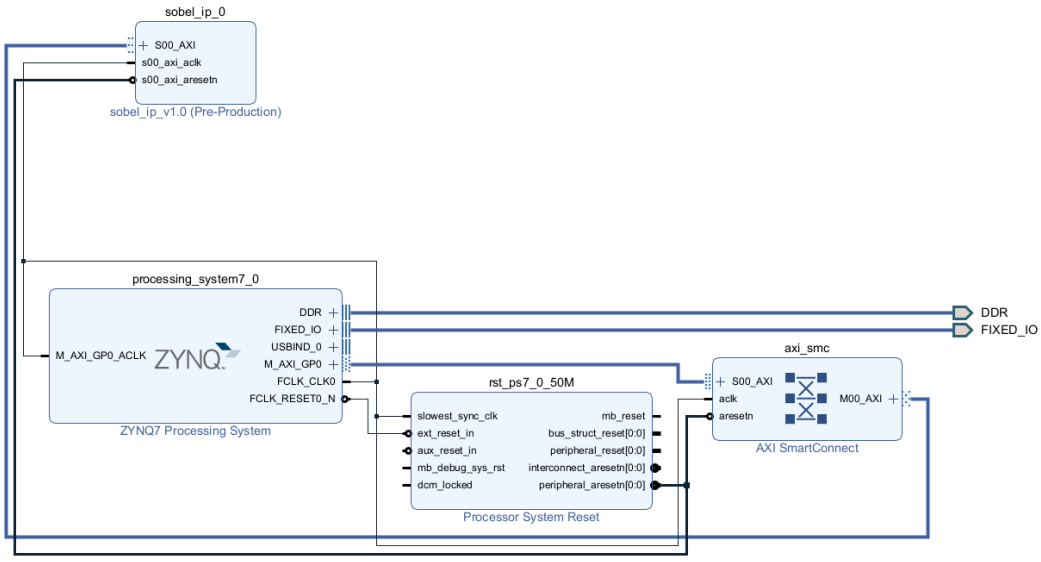


Figure 13 : Sobel Block Diagram

Once the IP block diagram is verified, a Hardware Design Language (HDL) wrapper is generated allowing a bitstream to be built. With the builtstream, a Hardware design file (.xsa) can be exported and used in the Vitis IDE environment to build the embedded project.

Within the embedded project platform, the source C code is updated to utilize the header files that contains macros and functions needed for reading and writing to the hardware registers connected to the IP design, as well as access tools for the SD card. Due to the dynamic memory allocation in relation to the BMP file, the stack and heap memory sizes are increased from their default sizes. In order to access the BMP file the SD cards drive needs to be mounted as shown in Figure 14.

```
    init_platform();
    FRESULT res;
    TCHAR *Path="0/";

print("Program Start\n");
inputFile="eye_bw.bmp";
if (inputFile == NULL){
    xil_printf( "ERROR: No input file specified!\n\n");
  return XST_FAILURE;
}

res=f_mount(&fatfs,Path,1);
if(res != FR_OK){
    xil_printf("Error:Card mount failed(%d)\n\r",res);
    return XST_FAILURE;
}
```

Figure 14 : SD Card Mount

Once the BMP data is read into the data structure from the SD card, it is cloned in to an identical structure. As for loops iterate through the pixel array, the registers that point to the IP block are written to as shown in Figure 15.

```
462         Xil_Out32(PIXEL_0_0_ADDR,pixel_0_0);
463         Xil_Out32(PIXEL_1_0_ADDR,pixel_1_0);
464         Xil_Out32(PIXEL_2_0_ADDR,pixel_2_0);
465         Xil_Out32(PIXEL_0_1_ADDR,pixel_0_1);
466         Xil_Out32(PIXEL_1_1_ADDR,pixel_1_1);
467         Xil_Out32(PIXEL_2_1_ADDR,pixel_2_1);
468         Xil_Out32(PIXEL_0_2_ADDR,pixel_0_2);
469         Xil_Out32(PIXEL_1_2_ADDR,pixel_1_2);
470         Xil_Out32(PIXEL_2_2_ADDR,pixel_2_2);
471         Xil_Out32(START_REG_ADDR,1);
472         pixel_start_reg = Xil_In32(START_REG_ADDR);
473         pixel_done_reg =Xil_In32(PIXEL_DONE_ADDR);/*
```

Figure 15 : IP Address Writing

A while loop task is used to determine when the sobel filter is done. The address locating the done signal is polled and compared to the mask.

```
89 #define DONE_READY_MASK 0x00000001
90
91 void wait_for_done_signal(){
92     while((Xil_In32(PIXEL_DONE_ADDR) & DONE_READY_MASK)==0){
93     }
94 }
```

Figure 16: Done Polling

Once the done signal is read, the pixel out register value is then saved to respective location of the destination pixel structure.

```
410         Xil_Out32(PIXEL_0_0_ADDR,pixel_0_0);
411         Xil_Out32(PIXEL_1_0_ADDR,pixel_1_0);
412         Xil_Out32(PIXEL_2_0_ADDR,pixel_2_0);
413         Xil_Out32(PIXEL_0_1_ADDR,pixel_0_1);
414         Xil_Out32(PIXEL_1_1_ADDR,pixel_1_1);
415         Xil_Out32(PIXEL_2_1_ADDR,pixel_2_1);
416         Xil_Out32(PIXEL_0_2_ADDR,pixel_0_2);
417         Xil_Out32(PIXEL_1_2_ADDR,pixel_1_2);
418         Xil_Out32(PIXEL_2_2_ADDR,pixel_2_2);
419         Xil_Out32(START_REG_ADDR,1);
420
421         wait_for_done_signal();
422         Xil_Out32(START_REG_ADDR,0);
423
424         pixel_out_reg = Xil_In32(PIXEL_OUT_ADDR);
425         pixel_done_reg =Xil_In32(PIXEL_DONE_ADDR);
426
427         dst_px[0] = pixel_out_reg;
```

Figure 17 : Register Calls

Once the loop iterations are complete the destination BMP structure is saved to the SD card.

```
//Apply filter
if(applyFilter(&srcImg,&dstImg) != 0){
      bmpFree(&dstImg);
      bmpFree(&srcImg);
      return XST_FAILURE;
}


xil_printf("Result: %dx%d\n", dstImg.width, dstImg.height);

if (bmpSave(outputFile, &dstImg) != 0)
  {
    bmpFree(&srcImg);
    bmpFree(&dstImg);
    return XST_FAILURE;
  }
bmpFree(&dstImg);


bmpFree(&srcImg);
print("Done!\n");
cleanup_platform();
```

Figure 18 : Destination Image Save

Results

The Verilog code is verified through a self-checking randomized input stimulus testbench. The stimulus is an 8-bit 9x1 array, meant to represent a matrix sample from an image. A golden value is calculated using initial integer values, and the square root value is saved to a real variable. The value is floored down to an 8-bit integer value needed to represent the integer pixel value.

```
int  g_x;
int  g_y;
real  g_real;
int g;

assign g_x = (-1*pixel_base_in[0]) + (-2*pixel_base_in[1]) + (-1*pixel_base_in[2]) + 1*pixel_base_in[6]
           + 2*pixel_base_in[7] + 1*pixel_base_in[8];
assign g_y = (-1*pixel_base_in[0]) + (-2*pixel_base_in[3]) + (-1*pixel_base_in[6]) + 1*pixel_base_in[2]
           + 2*pixel_base_in[5] + 1*pixel_base_in[8];
assign g_real = $sqrt((g_x * g_x)+(g_y * g_y));
assign g = $floor(g_real);
```

Figure 19 : Golden Model

The integer pixel value is clipped to a maximum value of 255 as that is the max pixel intensity value allowed for a pixel, shown in Figure 20.

```
function bit [7:0] sobel_out(
    input int g
);
if(g>255)
    return 8'(255);
else
    return 8'(g);
endfunction
```

Figure 20 : Max Pixel Value check

A snippet of the simulation results are shown below in Figure 21.

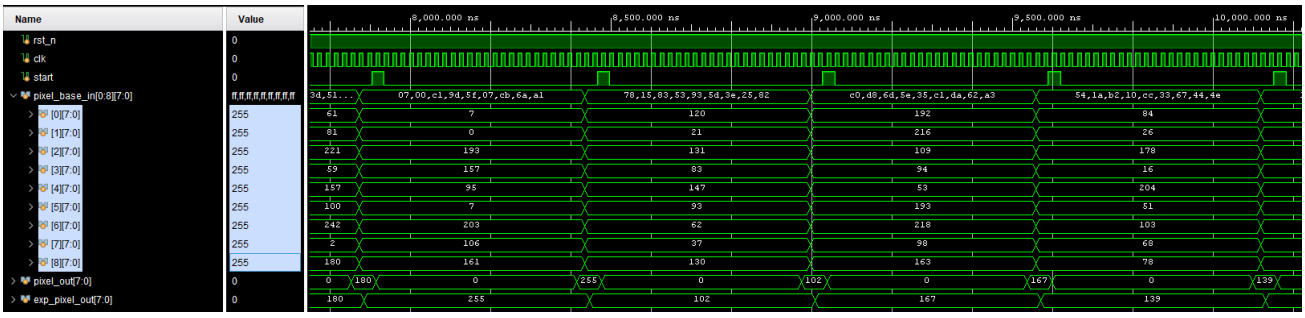


Figure 21 : Simulation Results

The address indexing was verified on a local C program to confirm the accessing method. A basic filter to apply the brightness was applied and called as a function to confirm the function method.

```
Loading: eye_bw.bmp
BMP header size 40
BMP data offset 1078
BMP image width 640
BMP image height 480
BMP bits per pixel 8
BMP Size of color table:  1024
Bytes per Pixel : 1
Size Pixel : 1
Num of colors: 256
RowSize: 640
Pixel Location:(1,480),Pixel value: 1
 |0 1 1|
 |0 1 1|
 |0 0 0|
Pixel Location:(2,480),Pixel value: 1
 |1 1 1|
 |1 1 1|
 |0 0 0|
Pixel Location:(3,480),Pixel value: 1
 |1 1 1|
 |1 1 1|
 |0 0 0|
Pixel Location:(4,480),Pixel value: 1
 |1 1 1|
 |1 1 1|
 |0 0 0|
Pixel Location:(5,480),Pixel value: 1
 |1 1 1|
 |1 1 1|
 |0 0 0|
```

Figure 22 : Test Log Output

Once loaded on to the hardware the UART COM port is outputted to the Vitis serial terminal to output information being relayed from the bare-metal application.

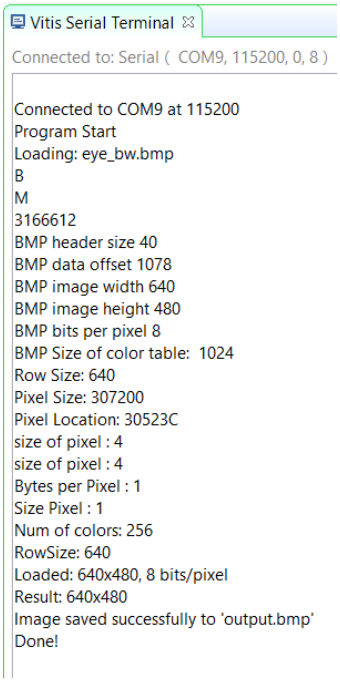


Figure 23 : UART Response

The sobel filter operation is shown having been applied on Figure 24, the source image, with the outputted image being Figure 25.

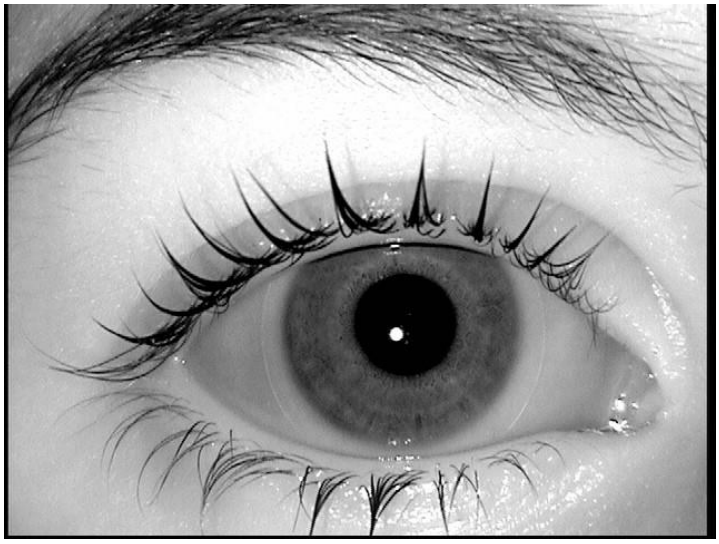


Figure 24 : Source Image

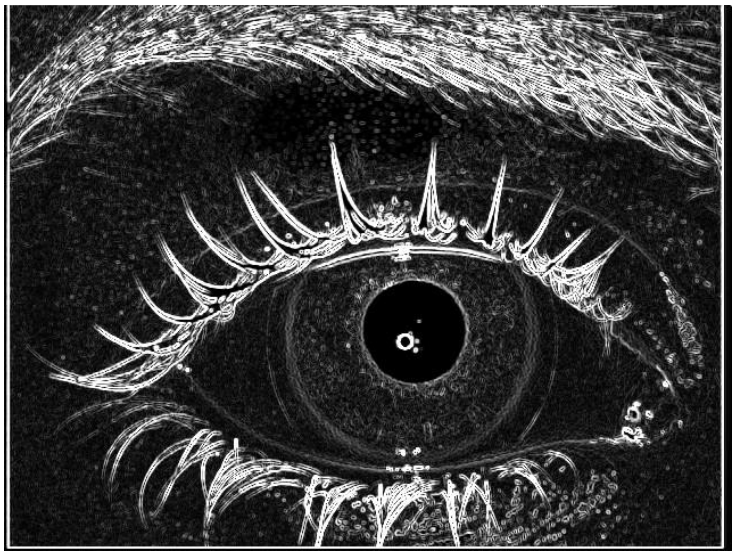


Figure 25 : Sobel Filter Applied Image

References

1. Zybo Z7 Reference Manual - Digilent Reference
2. https://yamin.cis.k.hosei.ac.jp/papers/ICCD96.pdf
3. https://verilogcodes.blogspot.com/2020/12/synthesizable-clocked-square-root.html
4. https://github.com/subrata05/image-processing-in-c/